СТРУКТУРА ПРОЕКТА

```
├── README.md
├── effects.js
├── favicon.png
├── index.html
├── script.js
└── style.css
```

ИСХОДНЫЙ КОД ФАЙЛОВ

=== ФАЙЛ: README.md ===

□ AstroTrails Pro Studio

> Мощный и автономный веб-редактор для создания фотографий звездных треков в режиме «Кометы» (Comet Mode) прямо из ваших таймлапс-видео.

□ [Запустить AstroTrails Pro Studio в браузере (GitHub Pages)](https://na1x-dev.github.io/AstroTrails/)

□ Основные особенности
- **Режим кометы нового поколения:** Использование нелинейных геометрических кривых для бесконечно длинного и плавного затухания хвостов звезд.
- **Экспорт в Оригинальном Качестве:** Рендеринг итогового изображения выполняется в 100% разрешении исходного видео (поддержка 4K UHD).
- **GPU-Оптимизация (Proxy-Preview):** Мгновенная цветокоррекция на 60 FPS за счет интеллектуального масштабирования превью-потока.
- **Интерактивное управление:** Регулировка ползунков с помощью кнопок точного шага `+` / `-` и колесика мыши.
- **Абсолютная автономность:** Работает полностью в браузере (Client-Side) без серверов, баз данных и внешних зависимостей.
- **Mobile Responsive:** Полная адаптация интерфейса для удобного использования на экранах смартфонов и планшетов.

□ Панель инструментов постобработки
- **Трассы:** Настройка крутизны экспоненциального затухания хвоста кометы.
- **Экспозиция:** Тонкая регулировка яркости и контрастности кадра.
- **Гамма:** Коррекция полутонов для создания глубокого, абсолютно черного ночного неба.
- **RGB микшер:** Раздельное управление Красным, Зеленым и Синим каналами для проявления туманностей и млечного пути.

□ Быстрый старт на локальном ПК
1. Склонируйте репозиторий:
   ```bash
   git clone https://github.com/Na1x-dev/AstroTrails.git
   ```
2. Откройте файл `index.html` в любом современном браузере.
3. Перетащите ваше склеенное видео астрофотографии в центр экрана!

=== ФАЙЛ: effects.js ===

```
const effects = {
    brightness: document.getElementById('brightness'),
    contrast: document.getElementById('contrast'),
    gamma: document.getElementById('gamma'),
    channelR: document.getElementById('channelR'),
    channelG: document.getElementById('channelG'),
    channelB: document.getElementById('channelB')
};

// Привязываем события и обновляем проценты по корректным ID
Object.keys(effects).forEach(key => {
    if (window.enhanceSlider) {
        window.enhanceSlider(effects[key]);
    }

    effects[key].addEventListener('input', () => {
        let suffix = '%'; if(key === 'gamma') suffix = '';
```

```
        const displayVal = key === 'gamma' ? (effects[key].value / 100).toFixed(1) : effects[key].value;

        // КЛЮЧЕВОЙ ФИКС: Точное совпадение ID с HTML-структурой
        const valElement = document.getElementById(`${key}Val`);
        if (valElement) {
            valElement.textContent = displayVal + suffix;
        }

        // Мгновенно применяем стили
        window.applyPostProcessing();
    });
});

window.resetAllEffects = function() {
    Object.keys(effects).forEach(key => {
        effects[key].value = 100;
        let suffix = '%'; if(key === 'gamma') suffix = '';
        const displayVal = key === 'gamma' ? '1.0' : '100';

        const valElement = document.getElementById(`${key}Val`);
        if (valElement) {
            valElement.textContent = displayVal + suffix;
        }
    });

    // Принудительно вызываем обновление холста
    if (window.applyPostProcessing) {
        window.applyPostProcessing();
    }
};

window.toggleFiltersState = function(isDisabled) {
    Object.keys(effects).forEach(key => {
        effects[key].disabled = isDisabled;
        const steps = effects[key].parentNode.querySelectorAll('.btn-step');
        steps.forEach(b => { b.disabled = isDisabled; });
    });
};

window.applyPostProcessing = function() {
    if (!window.astroState || !window.astroState.previewBuffer) return;

    const canvas = document.getElementById('glCanvas');
    const ctx = canvas.getContext('2d');

    const imgData = new ImageData(
        new Uint8ClampedArray(window.astroState.previewBuffer.data),
        window.astroState.previewBuffer.width,
        window.astroState.previewBuffer.height
    );
    const data = imgData.data;

    const b = parseInt(effects.brightness.value) / 100;
    const c = parseInt(effects.contrast.value) - 100;
    const g = parseInt(effects.gamma.value) / 100;
    const rM = parseInt(effects.channelR.value) / 100;
    const gM = parseInt(effects.channelG.value) / 100;
    const bM = parseInt(effects.channelB.value) / 100;

    const factor = (259 * (c + 255)) / (255 * (259 - c));

    for (let i = 0; i < data.length; i += 4) {
        let r = data[i] * b * rM;
        let gChan = data[i+1] * b * gM;
        let bl = data[i+2] * b * bM;

        if (c !== 0) {
            r = factor * (r - 128) + 128;
            gChan = factor * (gChan - 128) + 128;
            bl = factor * (bl - 128) + 128;
        }

        if (g !== 1) {
            r = 255 * Math.pow(Math.clamp(r, 0, 255) / 255, 1 / g);
```

```
            gChan = 255 * Math.pow(Math.clamp(gChan, 0, 255) / 255, 1 / g);
            bl = 255 * Math.pow(Math.clamp(bl, 0, 255) / 255, 1 / g);
        }

        data[i] = r < 0 ? 0 : (r > 255 ? 255 : r);
        data[i+1] = gChan < 0 ? 0 : (gChan > 255 ? 255 : gChan);
        data[i+2] = bl < 0 ? 0 : (bl > 255 ? 255 : bl);
    }

    ctx.clearRect(0, 0, canvas.width, canvas.height);

    const tempCanvas = document.createElement('canvas');
    tempCanvas.width = imgData.width;
    tempCanvas.height = imgData.height;
    tempCanvas.getContext('2d').putImageData(imgData, 0, 0);

    ctx.drawImage(tempCanvas, 0, 0, canvas.width, canvas.height);
};

Math.clamp = function(value, min, max) {
    return Math.min(max, Math.max(min, value));
};

document.getElementById('downloadBtn').addEventListener('click', (e) => {
    e.stopImmediatePropagation();
    if (!window.astroState || !window.astroState.rawCompositeBuffer) return;

    const originalBuffer = window.astroState.rawCompositeBuffer;
    const exportCanvas = document.createElement('canvas');
    exportCanvas.width = originalBuffer.width;
    exportCanvas.height = originalBuffer.height;
    const exportCtx = exportCanvas.getContext('2d');

    const imgData = new ImageData(
        new Uint8ClampedArray(originalBuffer.data),
        originalBuffer.width,
        originalBuffer.height
    );
    const data = imgData.data;

    const b = parseInt(effects.brightness.value) / 100;
    const c = parseInt(effects.contrast.value) - 100;
    const g = parseInt(effects.gamma.value) / 100;
    const rM = parseInt(effects.channelR.value) / 100;
    const gM = parseInt(effects.channelG.value) / 100;
    const bM = parseInt(effects.channelB.value) / 100;
    const factor = (259 * (c + 255)) / (255 * (259 - c));

    for (let i = 0; i < data.length; i += 4) {
        let r = data[i] * b * rM;
        let gChan = data[i+1] * b * gM;
        let bl = data[i+2] * b * bM;

        if (c !== 0) {
            r = factor * (r - 128) + 128;
            gChan = factor * (gChan - 128) + 128;
            bl = factor * (bl - 128) + 128;
        }

        if (g !== 1) {
            r = 255 * Math.pow(Math.clamp(r, 0, 255) / 255, 1 / g);
            gChan = 255 * Math.pow(Math.clamp(gChan, 0, 255) / 255, 1 / g);
            bl = 255 * Math.pow(Math.clamp(bl, 0, 255) / 255, 1 / g);
        }

        data[i] = r < 0 ? 0 : (r > 255 ? 255 : r);
        data[i+1] = gChan < 0 ? 0 : (gChan > 255 ? 255 : gChan);
        data[i+2] = bl < 0 ? 0 : (bl > 255 ? 255 : bl);
    }

    exportCtx.putImageData(imgData, 0, 0);

    const link = document.createElement('a');
    link.download = `AstroTrails_HD_${Date.now()}.png`;
```

```
    link.href = exportCanvas.toDataURL('image/png');
    link.click();
});

window.toggleFiltersState(true);
```

=== ФАЙЛ: index.html ===

```
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0, user-scalable=no">
    <title>AstroTrails</title>
    <link rel="preconnect" href="https://googleapis.com">
    <link rel="preconnect" href="https://gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap" rel="stylesheet">
    <link rel="stylesheet" href="https://cloudflare.com">
    <link rel="stylesheet" href="style.css">
</head>
<body>

<div class="editor-layout">
    <!-- Панель инструментов -->
    <aside class="sidebar">
        <div class="sidebar-header">
            <img src="favicon.png" alt="Logo" class="app-logo-img" onerror="this.style.display='none';
document.getElementById('fallbackLogo').style.display='block';">
            <div>
                <h2>AstroTrails</h2>
            </div>
        </div>

        <div class="sidebar-content">
            <!-- Секция 1: Управление файлом -->
            <div class="section-card">
                <div class="section-title">Файл</div>
                <button class="btn btn-tonal" id="uploadBtnClick">
                    <i class="fa-solid fa-folder-open"></i>
                    Открыть видео...
                </button>
                <input type="file" id="videoInput" accept="video/*" style="display: none;">
                <div id="fileInfoText">Файл не выбран</div>
            </div>

            <!-- Секция 2: Алгоритм треков -->
            <div class="section-card disabled-control" id="secTracks">
                <div class="section-title">Настройки трасс</div>
                <div class="slider-group">
                    <div class="slider-header">
                        <span>Прозрачность хвоста</span>
                        <span class="val" id="opacityVal">20%</span>
                    </div>
                    <input type="range" id="startOpacity" min="1" max="99" value="20" disabled>
                </div>
                <button class="btn btn-primary" id="processBtn" disabled>
                    <i class="fa-solid fa-wand-magic-sparkles"></i>
                    Сгенерировать треки
                </button>
            </div>

            <!-- Секция 3: Цветокоррекция -->
            <div class="section-card disabled-control" id="secEffects">
                <div class="section-title">Постобработка кадра (GPU)</div>

                <div class="slider-group">
                    <div class="slider-header"><span>Яркость</span><span class="val" id="brightnessVal">100%</span></div>
                    <input type="range" id="brightness" min="0" max="200" value="100" disabled>
                </div>

                <div class="slider-group">
                    <div class="slider-header"><span>Контрастность</span><span class="val" id="contrastVal">100%</span></div>
                    <input type="range" id="contrast" min="0" max="200" value="100" disabled>
                </div>
```

```
            <div class="slider-group">
                <div class="slider-header"><span>Гамма</span><span class="val" id="gammaVal">1.0</span></div>
                <input type="range" id="gamma" min="20" max="250" value="100" disabled>
            </div>

            <div class="divider"></div>

            <div class="slider-group rgb-slider r-slider">
                <div class="slider-header"><span>Красный канал (R)</span><span class="val" id="channelRVal">100%</span></div>
                <input type="range" id="channelR" min="0" max="200" value="100" disabled>
            </div>

            <div class="slider-group rgb-slider g-slider">
                <div class="slider-header"><span>Зеленый канал (G)</span><span class="val" id="channelGVal">100%</span></div>
                <input type="range" id="channelG" min="0" max="200" value="100" disabled>
            </div>

            <div class="slider-group rgb-slider b-slider">
                <div class="slider-header"><span>Синий канал (B)</span><span class="val" id="channelBVal">100%</span></div>
                <input type="range" id="channelB" min="0" max="200" value="100" disabled>
            </div>
        </div>
    </div>

    <div class="sidebar-footer">
        <button class="btn btn-tonal" id="resetBtn" disabled style="margin-bottom: 8px;">
            <i class="fa-solid fa-arrow-rotate-left"></i>
            Сбросить настройки
        </button>
        <button class="btn btn-primary" id="downloadBtn" disabled>
            <i class="fa-solid fa-download"></i>
            Экспорт в PNG
        </button>
    </div>
</aside>

<!-- Центральная область просмотра -->
<main class="viewport" id="dropZone">
    <div class="canvas-container">
        <canvas id="outputCanvas" style="display:none;"></canvas>
        <canvas id="glCanvas"></canvas>

        <div class="viewport-overlay" id="statusOverlay">
            <div class="loader-spinner" id="loaderSpinner"></div>
            <i class="fa-solid fa-cloud-moon overlay-icon" id="statusIcon"></i>
            <div class="status-title" id="statusTitle">Пустой холст</div>
            <div class="status-desc" id="statusText">Перетащите видео в центр экрана или используйте кнопку меню. На
мобильных — просто тапните по кнопке «Открыть видео».</div>

            <div class="progress-container" id="progressContainer">
                <div class="progress-bar" id="progressBar"></div>
            </div>
        </div>
    </div>
</main>
</div>

<script src="script.js"></script>
<script src="effects.js"></script>
</body>
</html>
```

=== ФАЙЛ: script.js ===

```
window.astroState = {
    videoFile: null,
    cachedFrames: [],
    rawCompositeBuffer: null, // Полноразмерный чистый буфер треков
    previewBuffer: null       // Быстрый уменьшенный буфер для предпросмотра
};

const videoInput = document.getElementById('videoInput');
const uploadBtnClick = document.getElementById('uploadBtnClick');
```

```
const fileInfoText = document.getElementById('fileInfoText');
const dropZone = document.getElementById('dropZone');
const startOpacityInput = document.getElementById('startOpacity');
const opacityVal = document.getElementById('opacityVal');
const processBtn = document.getElementById('processBtn');
const downloadBtn = document.getElementById('downloadBtn');
const resetBtn = document.getElementById('resetBtn');

// glCanvas теперь выступает как основной видимый 2D холст
const canvas = document.getElementById('glCanvas');
const ctx = canvas.getContext('2d');

function enhanceSlider(input, onInputCallback) {
    const wrapper = document.createElement('div');
    wrapper.className = 'slider-controls';
    input.parentNode.insertBefore(wrapper, input);

    const btnMinus = document.createElement('button');
    btnMinus.className = 'btn-step';
    btnMinus.innerHTML = '-';
    btnMinus.type = 'button';

    const btnPlus = document.createElement('button');
    btnPlus.className = 'btn-step';
    btnPlus.innerHTML = '+';
    btnPlus.type = 'button';

    wrapper.appendChild(btnMinus);
    wrapper.appendChild(input);
    wrapper.appendChild(btnPlus);

    const changeVal = (delta) => {
        if (input.disabled) return;
        const min = parseFloat(input.min);
        const max = parseFloat(input.max);
        let val = parseFloat(input.value) + delta;
        if (val < min) val = min;
        if (val > max) val = max;
        input.value = val;
        input.dispatchEvent(new Event('input'));
    };

    btnMinus.addEventListener('click', () => changeVal(-1));
    btnPlus.addEventListener('click', () => changeVal(1));

    wrapper.addEventListener('wheel', (e) => {
        if (input.disabled) return;
        e.preventDefault();
        changeVal(e.deltaY < 0 ? 1 : -1);
    }, { passive: false });
}

// Инициализируем улучшение для слайдера трасс
enhanceSlider(startOpacityInput);

startOpacityInput.addEventListener('input', (e) => {
    opacityVal.textContent = e.target.value + '%';
    if (window.astroState.cachedFrames.length > 0) renderCompositeTracks();
});

resetBtn.addEventListener('click', () => {
    startOpacityInput.value = 20;
    opacityVal.textContent = '20%';
    if (window.resetAllEffects) window.resetAllEffects(); // Сбросит фильтры в effects.js и перерисует холст
    renderCompositeTracks(); // Пересчитает длину треков звезд под 20%
});


['dragenter', 'dragover'].forEach(name => {
    dropZone.addEventListener(name, (e) => { e.preventDefault(); dropZone.classList.add('dragover'); });
});
['dragleave', 'drop'].forEach(name => {
    dropZone.addEventListener(name, (e) => { e.preventDefault(); dropZone.classList.remove('dragover'); });
});
```

```
dropZone.addEventListener('drop', (e) => { e.preventDefault(); dropZone.classList.remove('dragover'); if (e.dataTransfer.files.length)
handleFiles(e.dataTransfer.files[0]); });
uploadBtnClick.addEventListener('click', () => videoInput.click());
videoInput.addEventListener('change', (e) => { if (e.target.files.length) handleFiles(e.target.files[0]); });

function handleFiles(files) {
    if (!files || files.length === 0) return;

    // ИСПРАВЛЕНИЕ: Извлекаем первый конкретный файл из коллекции FileList,
    // независимо от того, пришел он через клик или через Drag & Drop
    const file = files instanceof FileList ? files[0] : files;

    // ПРОВЕРКА ФОРМАТА: Проверяем MIME-тип или расширение файла
    const isVideo = file.type.startsWith('video/') ||
                    file.name.toLowerCase().endsWith('.mp4') ||
                    file.name.toLowerCase().endsWith('.mov') ||
                    file.name.toLowerCase().endsWith('.avi');

    if (!isVideo) {
        // Выводим ошибку в центральный оверлей
        const icon = document.getElementById('statusIcon');
        icon.style.display = 'block';
        icon.className = "fa-solid fa-circle-exclamation overlay-icon";
        icon.style.color = "#f87171"; // Красный цвет ошибки

        document.getElementById('statusTitle').textContent = "Неподдерживаемый формат";
        document.getElementById('statusText').textContent = `Файл "${file.name}" не является видео. Пожалуйста, загрузите
склеенный таймлапс в формате MP4, MOV или AVI.`;
        document.getElementById('statusOverlay').classList.remove('hidden');

        // Блокируем инструменты
        fileInfoText.textContent = "Ошибка загрузки";
        secTracks.classList.add('disabled-control');
        startOpacityInput.disabled = true;
        processBtn.disabled = true;
        return;
    }

    // ЕСЛИ ВСЁ ХОРОШО — загружаем видеофайл
    window.astroState.videoFile = file;
    fileInfoText.textContent = file.name;

    if(window.astroState.cachedFrames.length) {
        window.astroState.cachedFrames.forEach(bitmap => {
            if (bitmap && typeof bitmap.close === 'function') bitmap.close();
        });
    }
    window.astroState.cachedFrames = [];
    window.astroState.rawCompositeBuffer = null;
    window.astroState.previewBuffer = null;

    // Сбрасываем цвет иконки к дефолтному
    document.getElementById('statusIcon').style.color = "";

    secTracks.classList.remove('disabled-control');
    startOpacityInput.disabled = false;
    processBtn.disabled = false;
    resetBtn.disabled = true;

    startOpacityInput.parentNode.querySelectorAll('.btn-step').forEach(b => b.disabled = false);

    const icon = document.getElementById('statusIcon');
    icon.style.display = 'block';
    icon.className = "fa-solid fa-circle-play overlay-icon";
    document.getElementById('statusTitle').textContent = "Видео готово к разбору";
    document.getElementById('statusText').textContent = "Нажмите кнопку «Сгенерировать треки», чтобы разбить таймлапс на
звездные дорожки.";
    document.getElementById('statusOverlay').classList.remove('hidden');
    downloadBtn.disabled = true;
    secEffects.classList.add('disabled-control');
    if (window.toggleFiltersState) window.toggleFiltersState(true);
}

processBtn.addEventListener('click', async () => {
```

```
    if (!window.astroState.videoFile) return;

    toggleUIBlock(true);
    document.getElementById('statusIcon').style.display = 'none';
    document.getElementById('loaderSpinner').style.display = 'block';
    document.getElementById('statusTitle').textContent = "Идет импорт кадров в кэш...";
    document.getElementById('progressContainer').style.display = 'block';
    document.getElementById('progressBar').style.width = '0%';

    const video = document.createElement('video');
    video.src = URL.createObjectURL(window.astroState.videoFile);
    video.muted = true; video.playsInline = true;

    video.onloadedmetadata = async () => {
        // Устанавливаем размер холста под видео
        canvas.width = video.videoWidth;
        canvas.height = video.videoHeight;

        const duration = video.duration;
        const fps = 25;
        const totalFrames = Math.floor(duration * fps);
        const step = duration / totalFrames;

        const tempCanvas = document.createElement('canvas');
        tempCanvas.width = canvas.width;
        tempCanvas.height = canvas.height;
        const tempCtx = tempCanvas.getContext('2d');

        window.astroState.cachedFrames = [];
        const maxAllowedFrames = 120;
        const frameSkip = Math.max(1, Math.floor(totalFrames / maxAllowedFrames));

        for (let i = 0; i < totalFrames; i += frameSkip) {
            video.currentTime = i * step;
            await new Promise(r => video.onseeked = r);

            tempCtx.drawImage(video, 0, 0, canvas.width, canvas.height);
            const bitmap = await createImageBitmap(tempCanvas);
            window.astroState.cachedFrames.push(bitmap);

            const percent = Math.round((i / totalFrames) * 100);
            document.getElementById('progressBar').style.width = `${percent}%`;
            document.getElementById('statusText').textContent = `Чтение потока: ${percent}% (Кадр ${i}/${totalFrames})`;
        }

        renderCompositeTracks();
    };
});

function renderCompositeTracks() {
    if (window.astroState.cachedFrames.length === 0) return;

    ctx.fillStyle = '#000000';
    ctx.fillRect(0, 0, canvas.width, canvas.height);

    const total = window.astroState.cachedFrames.length;

const sliderVal = parseFloat(startOpacityInput.value);
// Ограничиваем максимальную степень, чтобы она не уходила в бесконечность
const powerCoefficient = 0.1 + (sliderVal / 25);

// Внутри цикла рисования кадров:


    const factor = sliderVal / 100;


    // Экспоненциальная зависимость длины хвоста кометы


    // Временный холст для сборки треков
    const layerCanvas = document.createElement('canvas');
    layerCanvas.width = canvas.width;
    layerCanvas.height = canvas.height;
```

```
    const layerCtx = layerCanvas.getContext('2d');

    for (let i = 0; i < total; i++) {
        const progress = i / (total - 1 || 1);
        const opacity = Math.pow(progress, powerCoefficient);

        ctx.save();
        ctx.globalAlpha = opacity;
        ctx.globalCompositeOperation = 'lighten';
        ctx.drawImage(window.astroState.cachedFrames[i], 0, 0);
        ctx.restore();
    }

    // Сохраняем чистый полноразмерный буфер треков звезд
    window.astroState.rawCompositeBuffer = ctx.getImageData(0, 0, canvas.width, canvas.height);

    // СОЗДАЕМ ОПТИМИЗИРОВАННЫЙ ТРЕК-ПРЕВЬЮ (Прокси-картинка для быстрой цветокоррекции)
    const previewWidth = Math.min(canvas.width, 1024);
    const previewHeight = (canvas.height / canvas.width) * previewWidth;

    const pCanvas = document.createElement('canvas');
    pCanvas.width = previewWidth;
    pCanvas.height = previewHeight;
    const pCtx = pCanvas.getContext('2d');
    pCtx.drawImage(canvas, 0, 0, previewWidth, previewHeight);

    window.astroState.previewBuffer = pCtx.getImageData(0, 0, previewWidth, previewHeight);

    // Применяем фильтры
    if (window.applyPostProcessing) window.applyPostProcessing();

    document.getElementById('statusOverlay').classList.add('hidden');
    toggleUIBlock(false);
    secEffects.classList.remove('disabled-control');
    if (window.toggleFiltersState) window.toggleFiltersState(false);
    downloadBtn.disabled = false;
    resetBtn.disabled = false;
}

function toggleUIBlock(isBlocked) {
    processBtn.disabled = isBlocked;
    uploadBtnClick.disabled = isBlocked;
    startOpacityInput.disabled = isBlocked;
    dropZone.style.pointerEvents = isBlocked ? 'none' : 'auto';
    startOpacityInput.parentNode.querySelectorAll('.btn-step').forEach(b => b.disabled = isBlocked);

    if (isBlocked) {
        document.getElementById('loaderSpinner').style.display = 'block';
        resetBtn.disabled = true;
    } else {
        document.getElementById('loaderSpinner').style.display = 'none';
        document.getElementById('progressContainer').style.display = 'none';
    }
}
```

=== ФАЙЛ: style.css ===

```
:root {
    --bg-main: #0c0d10;
    --bg-sidebar: #13151a;
    --bg-card: #1c1e24;
    --bg-card-hover: #242730;
    --accent: #3b82f6;
    --accent-dim: rgba(59, 130, 246, 0.12);
    --text-main: #f3f4f6;
    --text-muted: #9ca3af;
    --border-color: #262930;
    --radius-lg: 12px;
    --radius-md: 8px;
}

* { box-sizing: border-box; margin: 0; padding: 0; font-family: 'Inter', sans-serif; -webkit-font-smoothing: antialiased; }
body { background-color: var(--bg-main); color: var(--text-main); overflow: hidden; height: 100vh; width: 100vw; }
```

```
.editor-layout {
    display: grid;
    grid-template-columns: 340px 1fr;
    height: 100vh;
    width: 100vw;
}

.sidebar {
    background-color: var(--bg-sidebar);
    border-right: 1px solid var(--border-color);
    display: flex;
    flex-direction: column;
    height: 100vh;
    order: 1;
}

.sidebar-header {
    padding: 16px 20px;
    border-bottom: 1px solid var(--border-color);
    display: flex;
    align-items: center;
    justify-content: center;
    gap: 10px;
}
.app-logo-img {  width: 48px; height: 48px; object-fit: contain; border-radius: var(--radius-md); }
.logo-icon { font-size: 24px; color: var(--accent); }
.sidebar-header h2 { font-size: 32px; font-weight: 400; letter-spacing: -0.3px; }
.version { font-size: 10px; color: var(--text-muted); background: #262930; padding: 2px 6px; border-radius: 4px; display: inline-block;
margin-top: 4px; }

#fileInfoText { font-size: 11px; color: var(--text-muted); margin-top: 8px; text-align: center; overflow: hidden; text-overflow: ellipsis; white-
space: nowrap; }
.sidebar-content { flex: 1; overflow-y: auto; padding: 20px; display: flex; flex-direction: column; gap: 16px; }
.section-card { background-color: var(--bg-card); border: 1px solid var(--border-color); border-radius: var(--radius-lg); padding: 16px;
transition: opacity 0.2s ease; }
.section-card.disabled-control { opacity: 0.25; pointer-events: none; }
.section-title { font-size: 11px; font-weight: 700; text-transform: uppercase; letter-spacing: 0.5px; color: var(--text-muted); margin-
bottom: 14px; }

.slider-group { margin-bottom: 14px; }
.slider-group:last-child { margin-bottom: 0; }
.slider-header { display: flex; justify-content: space-between; font-size: 12px; margin-bottom: 8px; color: var(--text-main); }
.slider-header .val { color: var(--accent); font-weight: 600; font-variant-numeric: tabular-nums; }
.slider-controls { display: flex; align-items: center; gap: 10px; }

.btn-step {
    background: #262930; border: 1px solid var(--border-color); color: var(--text-main); width: 32px; height: 32px; border-radius: 6px;
    cursor: pointer; display: flex; align-items: center; justify-content: center; font-size: 14px; font-weight: 700; transition: all 0.15s ease;
}
.btn-step:hover:not(:disabled) { background: #3b82f6; border-color: #3b82f6; color: #fff; }
.btn-step:disabled { opacity: 0.2; cursor: not-allowed; }

input[type="range"] { flex: 1; -webkit-appearance: none; height: 8px; background: #2d3139; border-radius: 4px; outline: none; cursor:
pointer; }
input[type="range"]::-webkit-slider-thumb {
    -webkit-appearance: none; width: 18px; height: 18px; border-radius: 50%; background: #ffffff; cursor: pointer; border: 3px solid var(--
bg-card);
    box-shadow: 0 2px 4px rgba(0,0,0,0.3); transition: transform 0.1s, background-color 0.1s;
}
input[type="range"]::-webkit-slider-thumb:hover { transform: scale(1.15); background: var(--accent); }

.rgb-slider.r-slider .val { color: #f87171; }
.rgb-slider.g-slider .val { color: #4ade80; }
.rgb-slider.b-slider .val { color: #60a5fa; }
.divider { height: 1px; background: var(--border-color); margin: 16px 0; }

.btn { display: inline-flex; align-items: center; justify-content: center; gap: 8px; height: 42px; padding: 0 16px; border-radius: var(--radius-
md); font-size: 13px; font-weight: 600; border: none; cursor: pointer; transition: all 0.2s ease; width: 100%; }
.btn-primary { background-color: var(--accent); color: white; }
.btn-primary:hover:not(:disabled) { background-color: #2563eb; }
.btn-primary:disabled { background-color: #262930; color: var(--text-muted); cursor: not-allowed; }
.sidebar-footer { padding: 16px 20px; border-top: 1px solid var(--border-color); display: flex; flex-direction: column; background: var(--
bg-sidebar); }
.btn-tonal { background-color: #262930; color: var(--text-main); border: 1px solid var(--border-color); }
```

```
.btn-tonal:hover:not(:disabled) { background-color: #2d3139; border-color: #4b5563; }
.btn-tonal:disabled { opacity: 0.2; cursor: not-allowed; }

.viewport { background-color: var(--bg-main); display: flex; justify-content: center; align-items: center; padding: 24px; height: 100vh; position: relative; transition: background-color 0.2s ease; order: 2; }
.viewport.dragover { background-color: #111622; }
.viewport.dragover .canvas-container { border: 2px dashed var(--accent); }
.canvas-container { max-width: 100%; max-height: 100%; position: relative; border-radius: var(--radius-lg); box-shadow: 0 20px 40px rgba(0,0,0,0.5); overflow: hidden; background-color: #050507; display: flex; justify-content: center; align-items: center; border: 2px solid transparent; transition: border 0.2s ease; }
canvas { max-width: 100%; max-height: 85vh; object-fit: contain; display: block; background: #000; }

.viewport-overlay { position: absolute; top: 0; left: 0; width: 100%; height: 100%; background: #111317; display: flex; flex-direction: column; justify-content: center; align-items: center; padding: 32px; text-align: center; z-index: 100; }
.viewport-overlay.hidden { display: none !important; }
.overlay-icon { font-size: 48px; color: var(--text-muted); margin-bottom: 16px; }
.status-title { font-size: 16px; font-weight: 600; margin-bottom: 8px; }
.status-desc { font-size: 13px; color: var(--text-muted); max-width: 360px; line-height: 1.5; }
.progress-container { width: 260px; height: 4px; background: #262930; border-radius: 2px; margin-top: 20px; overflow: hidden; display: none; }
.progress-bar { height: 100%; width: 0%; background: var(--accent); transition: width 0.05s linear; }
.loader-spinner { width: 40px; height: 40px; border: 3px solid #262930; border-top-color: var(--accent); border-radius: 50%; animation: spin 0.8s linear infinite; margin-bottom: 16px; display: none; }
@keyframes spin { to { transform: rotate(360deg); } }

/* МЕДИА-ЗАПРОС ДЛЯ СМАРТФОНОВ (АДАПТИВ) */
@media (max-width: 768px) {
    body { overflow-y: auto; }
    .editor-layout { grid-template-columns: 1fr; grid-template-rows: auto 1fr; height: auto; overflow-y: auto; }
    .viewport { height: 50vh; padding: 12px; border-bottom: 1px solid var(--border-color); position: sticky; top: 0; z-index: 150; background: var(--bg-main); }
    canvas { max-height: 45vh; }
    .sidebar { height: auto; border-right: none; border-top: 1px solid var(--border-color); }
    .sidebar-header { padding: 12px 16px; }
    .app-logo-img { width: 44px; height: 44px; }
    .sidebar-content { padding: 12px; overflow-y: visible; }
    .sidebar-footer { position: sticky; bottom: 0; z-index: 200; box-shadow: 0 -8px 24px rgba(0,0,0,0.5); }
}
```